

Hands-on Workshop

Augment, Generate & *Evaluate* across Modalities — Additional Lecture 3 of 3

Ezequiel López-Rubio

Friday, September 25, 2026

Malmö University, Sweden

DA631E — Artificial Intelligence for Data Science

Today: you drive

The last two lectures gave you the tools, across **text**, **image**, **audio**, **tabular**:

- **Augmentation** — transform real data (paraphrase, img2img, DSP, SMOTE).
- **Synthetic generation** — create labelled data (LLM, diffusion, TTS).

New this time: we *evaluate* properly

You now have **Lecture 7**. So today we don't just *peek* — we use train/test splits, **F1**, **TSTR**, and **leakage** checks to decide whether the data actually helps.

Learning outcomes for today

By the end of the workshop you should be able to:

1. Apply LLM/diffusion/TTS augmentation and generation to a fresh task.
2. **Evaluate** the result with a proper held-out set (accuracy, macro-F1, TSTR).
3. Diagnose data-quality problems: label drift, duplicates, low diversity, **leakage**.
4. Reason about bias, privacy (GDPR) and when synthetic data is trustworthy.

Outline

1. Setup and the evaluation toolkit
2. Track A — Text: rescue and evaluate
3. Track B — Tabular: generate and TSTR
4. Track C — Image or Audio: end-to-end
5. Track D — Quality & leakage audit
6. Track E — Data-centric mini-challenge
7. Reflection and next steps

Setup and the evaluation toolkit

Setup (5 minutes)

1. Open the shared **starter Colab notebook** (link on the course page).
2. *File* → *Save a copy in Drive* — work in **your** copy.
3. *Runtime* → *Change runtime type* → *T4 GPU* (free).
4. Run the first cell (installs + the `free_memory` helper from Lectures 1–2).

Keep it light

Small models, **tens** of examples, **one model at a time** — load it, reuse it across tracks, then `free_memory()` it before the next. If Colab stalls: *Runtime* → *Disconnect and delete runtime*, then reconnect.

Time budget (~90–105 min)

Segment	Time
Setup + warm-up + evaluation recap	15 min
Track A — Text: rescue & <i>evaluate</i>	20 min
Track B — Tabular: generate & <i>TSTR</i>	15 min
Track C — Image <i>or</i> Audio: end-to-end	20 min
Track D — Quality & leakage audit	15 min
Track E — Data-centric mini-challenge	15 min
Reflection & discussion	10 min

Do the tracks in order or jump to what interests you. Aim to finish **at least two**.

The evaluation toolkit (from Lecture 7) we use today

- **Split before anything:** *train* / *test* (and validation if tuning).
- **Augment / generate only the training side** — never the test set.
- **Metrics:** accuracy, **macro-F1** (better under imbalance), confusion matrix.
- **TSTR:** train on synthetic, test on *real* held-out data.
- **Leakage check:** no train item (or its variant) may appear in test.

*Same fixed model everywhere — so differences come from the **data**.*

Warm-up (10 min): one generate + one evaluate

```
from transformers import pipeline
import torch

gen = pipeline("text-generation", model="Qwen/Qwen2.5-0.5B-Instruct",
               device_map="auto")
msgs = [{"role": "user", "content": "Write one short negative review of a printer."}]
print(gen(msgs, max_new_tokens=50, do_sample=True, temperature=0.9)[0]
      ["generated_text"][-1]["content"])

from sklearn.metrics import f1_score
y_true, y_pred = ["pos", "neg", "pos", "neg"], ["pos", "neg", "neg", "neg"]
print("macro-F1:", round(f1_score(y_true, y_pred, average="macro"), 3))
```

Confirms your toolkit works: a generator *and* a metric from Lecture 7.

Track A — Text: rescue and
evaluate

Scenario

A support-ticket set is **imbalanced**: many “billing”, few “bug”. A classifier ignores the rare class.

Mission

1. Split real data into **train/test** *first*.
2. LLM-augment **only the training** minority class.
3. Report **macro-F1** on the *same* test set, with vs. without augmentation.

Track A — evaluate the effect honestly

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import f1_score

Xtr, Xte, ytr, yte = train_test_split(texts, labels, test_size=0.4,
                                     stratify=labels, random_state=0)

def macroF1(train_texts, train_labels):
    clf = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=1000))
    clf.fit(train_texts, train_labels)
    return f1_score(yte, clf.predict(Xte), average="macro")

print("baseline :", round(macroF1(Xtr, ytr), 3))
# ... build aug_texts/aug_labels by augmenting ONLY minority items in Xtr ...
print("augmented :", round(macroF1(Xtr + aug_texts, ytr + aug_labels), 3))
```

The test set is untouched — the comparison is now fair (Lecture 7 discipline).

Track B — Tabular: generate and TSTR

Scenario

You are prototyping a churn model but hold **little real data**. Generate a synthetic table and test whether it transfers to reality.

Mission

1. Generate ~40 validated JSON rows (schema from Lecture 2).
2. Keep a **real** held-out test set aside.
3. Run **TSTR**: train on synthetic, evaluate on real; report macro-F1.

Track B — proper TSTR

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, confusion_matrix

feat = ["age", "tenure_months", "tickets"]
# synth_df: LLM-generated + validated ; real_df: a small REAL held-out set
clf = RandomForestClassifier(n_estimators=100, random_state=0)
clf.fit(synth_df[feat], synth_df["churned"]) # TRAIN on synthetic

pred = clf.predict(real_df[feat]) # TEST on real
print("TSTR macro-F1:", round(f1_score(real_df["churned"], pred,
                                     average="macro"), 3))
print(confusion_matrix(real_df["churned"], pred))

free_memory(gen) # done with the text LLM (Tracks A-B); free before Track C
```

A good TSTR score is evidence the synthetic table captured real structure.

Track C — Image or Audio: end-to-end

Track C — pick one modality

Mission. Generate a tiny **2-class** set, then train + evaluate a simple classifier.

Image sd-turbo text→image (e.g. apple vs. banana), few images/class.

Audio SpeechT5 TTS (e.g. “yes” vs. “no”), few clips/class.

Evaluate

Hold out some generated items as a test split, or (better) test on a few **real** items.
Report accuracy and a confusion matrix.

First `free_memory(gen)` (done at the end of Track B), then load the image *or* audio model here.

Track C — image example (light features + classifier)

```
import numpy as np
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

def features(pil):
    # cheap 48-D colour histogram
    a = np.asarray(pil.resize((64, 64))) / 255.0
    return np.concatenate([np.histogram(a[..., c], bins=16, range=(0, 1))[0]
                           for c in range(3)])

X = np.array([features(img) for img, _ in images]) # from Lecture 2
y = np.array([lab for _, lab in images])
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)
clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
print("accuracy:", accuracy_score(yte, clf.predict(Xte)))
```

Audio analogue: replace features with **MFCC** means (`librosa.feature.mfcc`).

Track D — Quality & leakage audit

Take any dataset from Tracks A–C and **attack its quality**:

- **Near-duplicates**: how much is really new?
- **Diversity**: vocabulary / class balance / attribute coverage.
- **Label drift**: hand-check a sample of 10.
- **Leakage**: does any train item (or variant) match a test item?

Track D — measure it

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# 1) near-duplicate rate within a text set
V = TfidfVectorizer().fit(train_texts + test_texts)
S = cosine_similarity(V.transform(train_texts))
np.fill_diagonal(S, 0)
print("near-dup share:", round(float((S > 0.8).any(1).mean()), 3))

# 2) train/test LEAKAGE: any train item almost identical to a test item?
L = cosine_similarity(V.transform(train_texts), V.transform(test_texts))
print("leaked train items:", int((L > 0.9).any(1).sum()))
```

Interpret: leakage inflates scores; near-duplicates inflate size, not signal.

Track E — Data-centric mini-challenge

Track E — friendly challenge

Task

Using *only* data you augment/generate, maximise **macro-F1** on a **shared, real, held-out test set** (provided at the start).

Rules

- Fixed model for everyone: Term Frequency - Inverse Document Frequency (TF-IDF) + Logistic Regression.
- You may change **only the data**: prompts, mix, filtering, balance, size.
- Keep it light; declare your final training-set size.

Track E — the fixed, fair scoring harness

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import f1_score

def score(train_df, test_df):          # test_df = shared REAL held-out set
    clf = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=1000))
    clf.fit(train_df["text"], train_df["label"])
    return f1_score(test_df["label"], clf.predict(test_df["text"]),
                    average="macro")

print("your macro-F1:", round(score(my_train_df, real_test_df), 3))
```

Same model for all: the **data** is the only variable — data-centric AI, evaluated by Lecture 7 rules.

Expect to discover, with real numbers this time:

- **Balanced** classes beat raw volume.
- **Diverse** phrasing generalises better to real text.
- Aggressive **filtering** of drifted/duplicate items pays off.
- A little **real seed** data mixed in goes a long way.

Better data > more data.

Reflection and next steps

Group reflection (10 min)

- Where did generated/augmented data **help** — and where did it **hurt**?
- Which quality problem was hardest to detect? Did **leakage** fool anyone?
- Would you trust synthetic data for a **privacy-sensitive** Swedish/EU project?
- Given Lecture 7, *how* would you convince a stakeholder it works?

Submit your Colab notebook containing

1. At least **two** completed tracks (A–E), spanning ≥ 2 **modalities**.
2. A proper **evaluation** for each: split, metric (macro-F1), and one leakage/quality check.
3. A short **reflection** (~150 words): what worked, what didn't, what you'd measure next.

Assessed on sound *evaluation* and insight — not on topping the leaderboard.

How this connects to the rest of the course

- **Lecture 7 (done)**: the evaluation discipline you used all day.
- **Lectures 8–9**: supervised learning — models to feed with this data.
- **Lecture 12**: neural networks; **Lecture 13**: what is *inside* the generators.
- **Mini-projects**: treat augmentation & synthetic data as tools in your pipeline.

Thank you — now go build, evaluate, and break some data!

Facilitator notes (optional)

- Provide a shared **real** test set per modality as small files (text CSV; a few images; a few WAVs).
- Pre-run the warm-up so model files cache on the shared runtime image.
- Insist on *split-first*; the most common bug is augmenting before splitting (leakage).
- If GPUs are scarce: `flan-t5-small` + fewer items run on CPU; skip diffusion/TTS or share outputs.
- Keep a live macro-F1 board for Track E to spark discussion, not competition.